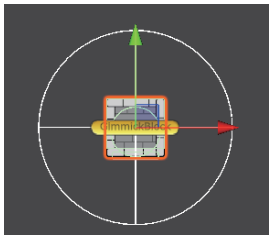
GimmickBlock



```
public float length = 0.0f;      // 自動落下検知距離
public bool isDelete = false;    // 何かに触れたら削除するフラグ
bool fade = false;              // 消滅開始フラグ
float fadeTime = 0.5f;           // フェードアウト時間
```

Start の主な挙動

物理的特性を一時封印（くっついている Dead オブジェクトも非 Active に）

```
// Rigidbody2Dの物理挙動を停止
Rigidbody2D rbody = GetComponent<Rigidbody2D>();
rbody.bodyType = RigidbodyType2D.Static;
```

Update の主な挙動

①プレイヤーとの距離計測をして変数 d に取得

```
// プレイヤーとの距離計測
float d = Vector2.Distance(transform.position, player.transform.position);
```

距離 d が想定 (length) より小さくなったら（近づいたら）

（1 回だけ）・物理特性を復活して下に落ちるようになる
・消していた Dead オブジェクトも Active に

```
if (rbody.bodyType == RigidbodyType2D.Static)
{
    // Rigidbody2Dの物理挙動を開始
    rbody.bodyType = RigidbodyType2D.Dynamic;
    deadObj.SetActive(true);     //死亡あたりを表示
}
```

②消滅開始フラグ (変数 fade) が true になり次第、毎フレーム少しずつ透明になり最後は自己抹消

・変数 fadeTime から、毎フレーム Time.deltaTime を引く
（徐々に 0 に近づいていく）

・そのフレーム開始時におけるカラー情報を SpriteRenderer コンポーネントから
変数 col に取得し、取得した col の透明度 α 値を fadeTime の減少にあわせて下げる。
下げた後は、SpriteRenderer のカラー情報として戻す。

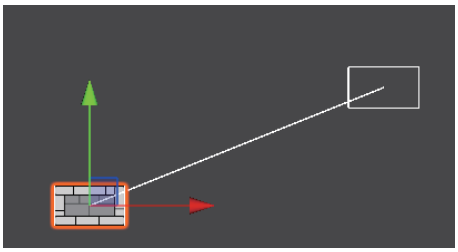
```
Color col = GetComponent<SpriteRenderer>().color;   // カラーを取り出す
col.a = fadeTime;   // 透明値を変更
GetComponent<SpriteRenderer>().color = col; // カラーを再設定する
```

・fadeTime が 0 以下になったら自オブジェクトごと Destroy で抹消

OnCollisionEnter2D（ブロックが何かと触れたときの挙動）

・もし isDelete フラグが true なら：何かに触れ次第、消滅開始フラグ (変数 fade) を true にする

MovingBlock



```
public float moveX = 0.0f;          //X移動距離
public float moveY = 2.0f;          //Y移動距離
public float times = 3.0f;          //片道の時間
public float wait = 0.0f;           //インターバルの時間
public bool isMoveWhenOn = false;   //乗ってから動くかどうかフラグ
public bool isCanMove = true;       //動作フラグ

Vector3 startPos;                   //初期位置
Vector3 endPos;                     //移動位置
bool isReverse = false;             //反対方向フラグ

float movep = 0;                    //移動補完値：進捗ともいえる/全体を1とした時の割合
```

変数 isMoveWhenOn が true なら「乗ると反応するブロック」
false なら「常に自動で動いているブロック」

Start の主な挙動

・スタート地点と、エンド地点の座標を計算

Update の主な挙動

※動作フラグ (変数 isCanMove) が true の間だけ動く

Vector2.Lerp(出発座標、到着座標、進捗率) で算出した座標 → そのフレームにおける MovingBlock の座標として代入

逆走フラグ (変数 isReverse) が false の時は往路（スタート　→　エンド）

```
transform.position = Vector2.Lerp(startPos, endPos, movep);
```

true の時は復路（エンド　→　スタート）

```
transform.position = Vector2.Lerp(endPos, startPos, movep);
```

進捗率は毎フレーム 増えていく（0→ Max1 に向けて微増）

5 秒で移動したい場合

0 座標　　10 座標

distance = 10（全体の距離）
ds = distance / times = 10 / 5 = 2 (1 秒あたり進むべき距離)
df = ds * Time.deltaTime = 2 * その 1 フレームにかかった時間 (仮に 0.02 秒)
= 0.04 (その 1 フレームあたりに進むべき距離)

movep += df / distance　→
movep += 0.04 / 10　→
movep = それまでの movep + 0.04 / 10　→ つまり
movep = それまでの進捗率 + その 1 フレームの時間で増加すべき進捗率
という意味になり、毎フレーム movep は微増していく

いずれ進捗率が 100%（値としては 1）になったら目的地についたという事

- 逆の道のりに備えて進捗率（変数 movep）をリセット
- 逆走フラグの (true/false) を逆転させておく
- とりあえず止める（変数 isCanMove) を false

```
movep = 0.0f;
isReverse = !isReverse;
Stop();
```

```
//移動フラグを下ろして止める
public void Stop()
{
    isCanMove = false;
}
```

変数 isMoveWhenOn が true なら「乗ると反応するブロック」なので自動では再開しない。
isMoveWhenOn が false の場合のみ「時間差で動作フラグ (変数 isCanMove) が true になる」

```
if (isMoveWhenOn == false)
{
    Invoke("Move", wait);
}
```

```
//移動フラグを立てることで動かす
public void Move()
{
    isCanMove = true;
}
```

OnCollisionEnter2D（ブロックと Player (collision) が触れたときの挙動）

「ブロックと Player」を一体化させる　→　Player の親オブジェクトをブロックにする

```
//プレイヤーの親を自分（床）にする
collision.transform.SetParent(transform);
```

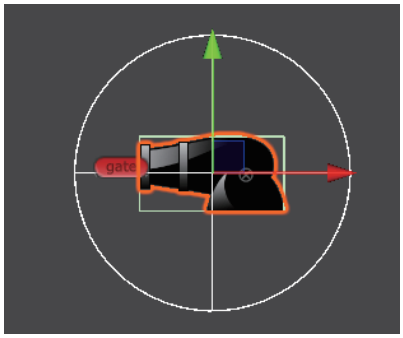
ちなみに変数 isMoveWhenOn が true なら「乗ると反応するブロック」なので
接触して初めて動作フラグ (変数 isCanMove) が true になる

OnCollisionEnter2D（ブロックと Player が離れたときの挙動）

「ブロックと Player」を一体化を解く　→　Player の親オブジェクトは何物でもない

```
//プレイヤーの親をなしにする（親子関係の解消）
collision.transform.SetParent(null);
```

CannonController




```
public GameObject objPrefab;            //発生させるPrefabデータ
public float delayTime = 3.0f;          //遅延時間
public float fireSpeed = 4.0f;          //発射速度
public float length = 8.0f;             //範囲

GameObject player;                      //プレイヤー
Transform gateTransform;                //発射口のTransform
float passedTimes = 0;                  //経過時間
```

Update の主な挙動

時間経過を計測

```
//待機時間加算
passedTimes += Time.deltaTime;
```

```
//Playerとの距離チェック
bool CheckLength(Vector2 targetPos)
{
    bool ret = false;
    float d = Vector2.Distance(transform.position, targetPos);
    if (length >= d)
    {
        ret = true;
    }
    return ret;
}
```

2 者間の直線距離を算出

基準値 (length) 内なら true

Player との距離が近く、かつ時間が基準を超えていれば

```
if (CheckLength(player.transform.position))
```

```
if (passedTimes > delayTime)
```

Instantiate メソッドにて Shell プレハブを gate オブジェクトの位置に生成（計測時間はリセット）

```
Vector2 pos = new Vector2(
    gateTransform.position.x,
    gateTransform.position.y);
GameObject obj = Instantiate(
    objPrefab,
    pos,
    Quaternion.identity);
```

生成した Shell プレハブ側の Rigidbody2D の AddForce メソッドで Shell を飛ばす

```
Rigidbody2D rbody = obj.GetComponent<Rigidbody2D>();
rbody.AddForce(v, ForceMode2D.Impulse);
```

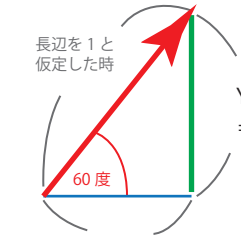
Cannon の向いている方向（Vector2) である変数 v

```
float angleZ = transform.localEulerAngles.z - 180;
float x = Mathf.Cos(angleZ * Mathf.Deg2Rad); //角度に対して底辺を取得
float y = Mathf.Sin(angleZ * Mathf.Deg2Rad); //角度に対して垂直辺を取得
Vector2 v = new Vector2(x, y) * fireSpeed;
```

Rotation X 0 Y 0 Z -120

角度 -120 の場合
絵が向いているのは 60 度
60 度に Shell を
飛ばしたい

角度でなく
Vector2
(x 成分と y 成分の指定)
で現わせないと
AddForce に
指定できない！
（困った！）



三角関数を使えば角度を
X 成分と Y 成分の割合に分解できる！

Y 軸は Sin(角度)
＝0.866

X 軸は Cos(角度)
＝0.5

方向データ
変数 v ＝new Vector2(変数 x , 変数 y)
＝new Vector2(0.5 , 0.866)

Deg2Rad（デグリートゥーラジアン）は、プログラミングや数学の計算において、**「度（Degree）」を「ラジアン（Radian）」に変換する定数または関数**のことです。

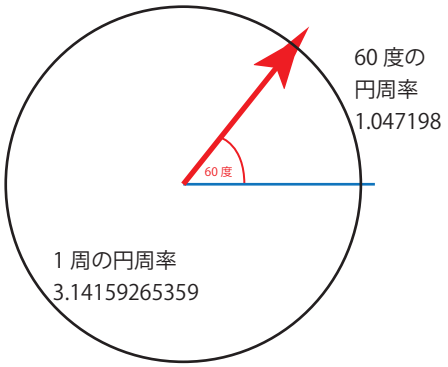
読み方

- **デグリートゥーラジアン** (Degree to Radian)

概要

- **役割**: 一般的な角度の単位「度」を、数学的な計算（三角関数など）で使われる「ラジアン」に直します。
- **計算式**: $1^\circ = \frac{\pi}{180}$ ラジアン です。
- **値**: $\frac{\pi}{180} \approx 0.0174533$ です。
- **用途**: Unity（Mathf.Deg2Rad）、PHP（deg2rad()）、Python/NumPy（deg2rad()）、MATLAB などで利用されます。

ラジアンとは弧度法つまり円周率による角度の捉え方



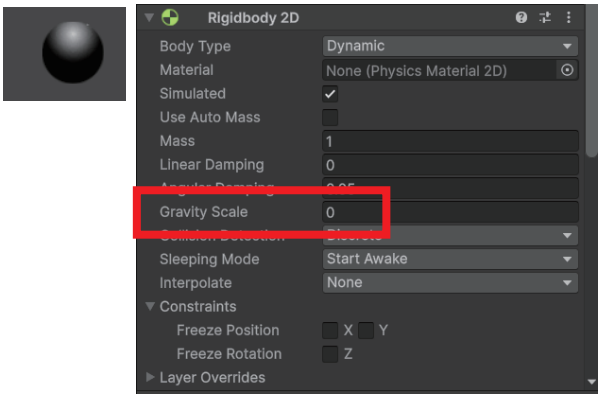
360 度　→　1 周　→　円周率 3.14
（例）
60 度 → 60 * Deg2Rad → 60 * 0.0174533 → 円周率 1.047198

Unity の三角関数のメソッド（Sin、Cos、Tan）は引数はラジアン形式

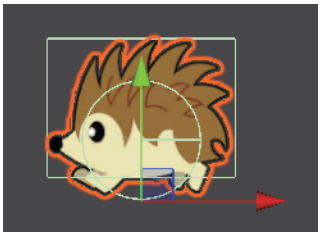
```
Mathf.Cos(angleZ * Mathf.Deg2Rad);
Mathf.Sin(angleZ * Mathf.Deg2Rad);
```

Shell プレハブ

重力なしでまっすぐ飛ぶ



EnemyController



```
public float speed = 3.0f;          // 移動速度
public bool isToRight = false;      // true=右向き　false=左向き
public float revTime = 0;           // 反転時間
public LayerMask groundLayer;       // 地面レイヤー
bool onGround = false;              // 地面フラグ
float time = 0;
```

Update の主な挙動

- 地上判定をしている
- 時間を計測して
  基準時間を超える度
  向きフラグを反転させる
  (変数 isToRight)

FixedUpdate の主な挙動

- 向きフラグに応じた speed 値を Velocity に与えている

OnTriggerEnter2D（Enemy が何かと触れたときの挙動）

- 時間計測をリセットする
- 向きフラグを反転させる